

Acoustic Tracer

GitHub Repository Link: [Acoustic Tracer](#)

Team Members:

- Alex Wright - Simulation Core / Front-end
- Patryk Mrozek - Simulation Core
- Eoghan Murphy - Simulation Core / Optimization
- Michael McCarthy - Front-end

Table of Contents

- Introduction

Our project ‘Acoustic Tracer’ is a three-dimensional, acoustic visualiser that allows users to visualise sound travelling throughout a modelled environment as a heatmap. At the core it is a C library, where the user can read in a `.glb` file (3D Model File), insert (a) speaker(s), specify simulation settings, and receive back a heatmap of how the sound travelled through the environment over time. Our final project extends it into a web-based application that renders this heatmap and makes the configuration of the scene and simulation more user-friendly, while also allowing more technical users to achieve their desired configuration. A user can create an account, upload `.glb` model files, view the models in a 3D scene-viewer, configure the scene and simulation settings, and finally run the simulation, returning a heatmap which they can play through and replay at a later time if desired. The aim of this project was to create a unique software that could be used by architects, acoustic specialists, or anyone who desires to model how sound travels through their environment. This was also a passion project to explore the potential and concept of ray-tracing, along with creating a software that people would actually use.

- Previous Works

On carrying out research into existing technologies before starting this project, we weren’t able to find an exact software to meet our needs.

- Overview of ours (basically the presentation)
- architecture (front & back)

Upon starting the project our first task was to set up the shared GitHub repository where our code was to be hosted. This was an involved process with the whole team, as we wanted to ensure a proper workflow with professional version control standards. We set up an organisation, as to not have a single person hosting the repository. We then created the repository **AcousticTracer** where we each forked it, giving ourselves a personal ‘copy’ of the repository to serve as our **origin**. We then added a remote **upstream** where we could merge changes from our personal forks to the original repository located in the organisation. This allowed us to each work independently on features in parallel

and then push these changes into a shared repository. We configured approval rules to ensure that members could not approve their own pull requests, and that each pull request required at least one approval before merging into the main branch.

Once the GitHub had been configured the next step was to work on the C library specification, since that is the core essence of the project. The C Library will henceforth be referred to as the ‘core’ of the project. The members working on the core of the project began at the ‘top’ of the core, creating the specification and outline of the public API presented by the core to the end user. Our initial scope of the project included a ‘command buffer’ that the output would be written to. The idea behind the command buffer was that the output of the simulation would be instructions for any graphics engine (OpenGL, Vulkan, etc.), on how to draw our heatmap. This was in the hopes to create a truly independent, graphics library agnostic library, but this was later scrapped due to the complexity. We settled on creating a consistent communication standard instead, that could be parsed by any front-end, or visualisation tool. This communication standard will be discussed at a later stage.

The architecture / workflow we envisioned for the user was as follows:

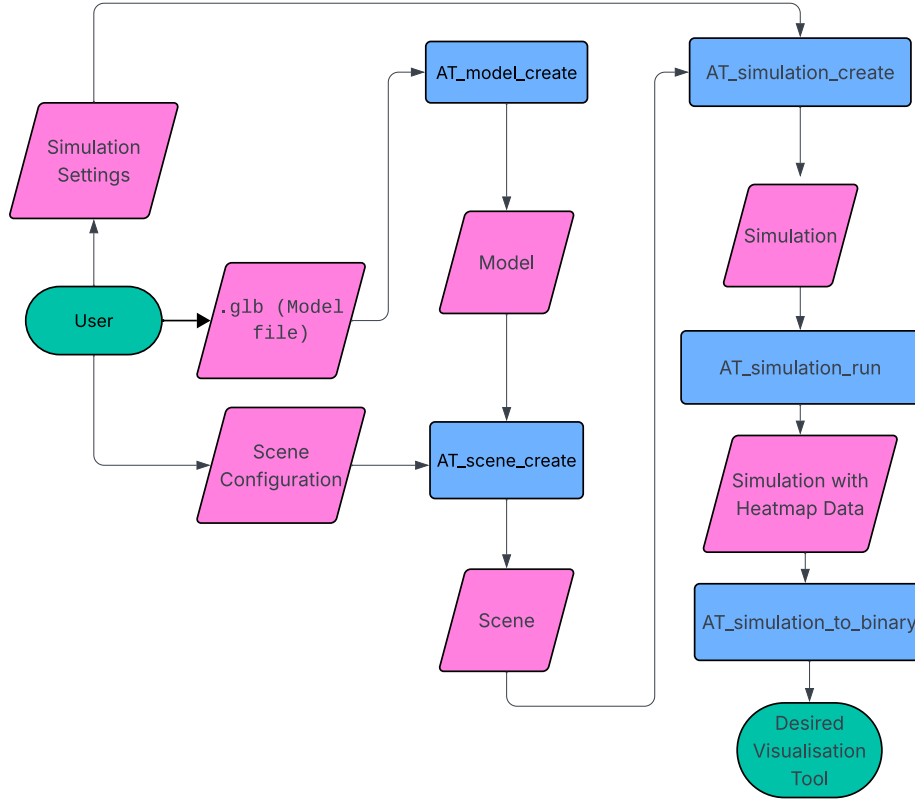


Figure 1: Flowchart of final architecture

The core of the project was written in C, over other languages, for a few key reasons. Firstly, C is extremely performant, which is crucial for our application since it is extremely computationally heavy. A key feature for us was the library `raylib` which C provides, and this allowed us to visualise our core during development. Finally we have been using dynamically typed languages such as Python and JavaScript during our degree, and we wanted to each improve our programming skills with a statically typed language. Since C is devoid of object-oriented features like classes and methods, the workflow for our C library must follow a certain structure. The user of the C library must create `struct` instances and pass these to functions that alter them. However users of the included front-end need not worry about the implementation.

The specification of the project involved the design of the following structures and data-types (each prefixed with `AT_` as part of our core library):

The front-end, as mentioned is our method of creating a universal visualiser for the heatmap of the model, while remaining platform agnostic (i.e. usable on MacOS, Linux, Windows, iOS, Android, etc.). Initially, during the project

specification phase, we designed a communication standard which we stuck with throughout the duration of the project, and it worked well. However, while the structure of the data remained the same, the data type had to change, which will be discussed later. We decided to create a server in C, that exposed an endpoint `/run` to the user, where they can send the scene and simulation config, and receive the heatmap data as the server response. The front-end, could then make a HTTP request to the server which is easily performed on any language.

C

- Implementation
 - Two phase simulation:
 - * Rays
 - Moller-Trumbmore alg, linked list/tree structure of rays...
 - * Voxels
 - DDA, energy bins design, voxel grid, attenuation, absorption...
- Design choices (c library)
 - `at.h`, opaque API, `at_internal.h`, returning `AT_Result`...
- Optimisation
- API comms (both, potent split into two sections)
 - bins

Frontend

Introduction and Motivation

The AcousticTracer project pairs a C simulation engine with a browser-based frontend, which configures, stores, and visualises the simulation. Tools like ODEON exist for this domain, but they are desktop-only, expensive, and closed-source. A browser-based alternative would be freely accessible, require no installation, and immediate real-time use. This section describes the architecture of that browser frontend: what its major components are, how they interact at runtime, and what was learned building this project while starting with little to no prior experience in Three.js, WebGL, or production React architecture.

Core Goals

The frontend has three responsibilities, each with distinct technical demands:

1. **Configure and submit** — The user uploads a `.gltf` 3D room model, sets simulation parameters (voxel size, ray count, FPS, surface material), and interactively places a sound source by clicking inside the 3D scene. The configuration is sent to the C backend as JSON.

2. **Decode and store** — The C backend returns simulation results as a custom binary format (`.atrb`). The frontend must decode this into typed arrays, store it in cloud file storage for persistent storage, and cache the parsed result so revisiting a completed simulation is instantaneous.
 3. **Render and replay** — The decoded frames must be visualised as a 3D voxel heatmap overlaid on the room model, animated at the simulation's original FPS. A typical simulation produces tens of thousands to hundreds of thousands of voxels, each requiring per-frame position and color updates. This must run at interactive frame rates in a browser, with a fully functioning playback system.
- Design choices (front)
 - state storing
 - storing (not state)
 - rendering
 - replaying
 - auth
 - routing
 - What we would do different (ODEON)
 - Lessons learned
 - Future Feature Plans
 - Conclusions [^1]
 - references

References